

Phase 3: Engineering Plan for PoC and Release

1. Objective

Phase 3 explains how to move the current Docker-based crawler from a working implementation into a small Proof of Concept and then toward a first cloud release.

The goal of the PoC is not to crawl billions of URLs immediately. The goal is to prove that the system can run end-to-end with a smaller dataset, produce correct metadata/content output, handle failures, and be deployed in the cloud.

2. Proof of Concept Scope

In Scope

- Run the crawler through Docker.
- Accept URL input from a text file.
- Optionally support MySQL input by `year` and `month`.
- Validate and normalize URLs.
- Generate URL hashes for deduplication.
- Crawl URLs and extract metadata.
- Store raw content and metadata.
- Track success, failure, retry, and skipped URLs.
- Deploy the Docker image to a cloud service.

Out of Scope for Initial PoC

- Billion-scale crawling.
- Full production Kafka/SQS architecture.
- Complex autoscaling.
- Multi-region deployment.
- Advanced cost optimization.
- Full monitoring platform with every dashboard.

The PoC should prove the core flow first. Scaling and optimization can come after the flow is stable.

3. Proposed PoC Flow

```
Input File / MySQL
  |
  v
Ingestion
validate + normalize + hash URLs
  |
  v
Crawler Container
fetch pages + extract metadata
  |
  v
Storage
raw content + metadata + errors
  |
  v
Logs / Metrics
progress + failures + latency
```

For local testing, Docker Compose is enough.

For cloud testing, a simple option is:

- AWS ECS Fargate / AWS App Runner, or
 - Google Cloud Run, or
 - Azure Container Apps.
-

4. Engineering Next Steps

Step 1: Stabilize Docker

Make sure the crawler runs consistently inside Docker.

Expected output:

```
docker build works
docker run works
docker compose up works
sample URLs are processed successfully
```

Estimate:

0.5 to 1 day

Step 2: Add Configuration

Move runtime settings into environment variables.

Examples:

```
BATCH_ID=2026-07-poc
INPUT_FILE=sample_urls.txt
OUTPUT_DIR=results/
CRAWL_TIMEOUT_SECONDS=20
MAX_RETRIES=3
```

Estimate:

0.5 day

Step 3: Add Basic Ingestion

The ingestion step should:

- Read URLs from a file.
- Optionally read URLs from MySQL.
- Validate URLs.
- Normalize URLs.
- Generate `normalized_url_hash`.
- Skip invalid and duplicate URLs.

Estimate:

1 to 2 days

Step 4: Improve Crawl Output

The crawler should produce consistent output for every URL.

Metadata should include:

- original URL
- normalized URL
- URL hash
- HTTP status code
- final URL
- title
- meta description
- content type
- content length
- crawl status
- error reason, if failed

Estimate:

1 to 2 days

Step 5: Add Retry and Error Handling

Add simple retry behavior for temporary failures.

Retryable examples:

- timeout
- DNS failure
- connection reset
- HTTP 429
- HTTP 500/502/503/504

Permanent examples:

- invalid URL
- HTTP 404
- HTTP 410
- unsupported content type

Estimate:

1 day

Step 6: Deploy to Cloud

Deploy the Docker image to a simple cloud container platform.

Recommended AWS PoC:

ECR for Docker image
ECS Fargate or App Runner for execution
S3 for raw content/output files
CloudWatch for logs

Recommended GCP PoC:

Artifact Registry for Docker image
Cloud Run for execution
GCS for raw content/output files
Cloud Logging for logs

Estimate:

2 to 4 days

5. Potential Blockers

Blocker	Risk	Mitigation	Estimate
Docker dependency issues	Low	Pin dependencies and test build locally	1-3 hours
Bad or malformed URLs	Low	Add URL validation and skip reasons	2-4 hours

Blocker	Risk	Mitigation	Estimate
Duplicate URLs	Low	Use normalized URL hash	2-4 hours
Some sites timeout or block crawling	Medium	Add timeout, retry, and rate limits	1 day
Output schema changes	Medium	Freeze metadata schema before PoC test	0.5 day
Cloud IAM/storage permissions	Medium	Use least-privilege role and test with small files	0.5-1 day
Cloud cost surprises	Medium	Set max workers, small test batches, and budgets	0.5 day
Missing observability	High	Add basic logs and counters before cloud test	1 day

6. Implementation Schedule

Minimal PoC Timeline

Day	Work
Day 1	Docker stabilization and environment configuration
Day 2	File ingestion, URL validation, normalization, hashing
Day 3	Metadata output, raw content output, error handling
Day 4	Retry logic and local Docker Compose test
Day 5	Cloud deployment setup and small cloud test

Minimum estimate:

5 working days

Higher Quality PoC Timeline

Time	Work
Week 1	Local Docker PoC, ingestion, metadata, retries, local test
Week 2	Cloud deployment, logging, metrics, evaluation report, release cleanup

Recommended estimate:

1.5 to 2 weeks

7. PoC Evaluation Plan

Test the system in stages:

Stage 1: 100 URLs
Stage 2: 1,000 URLs
Stage 3: 10,000 URLs
Stage 4: 100,000 URLs, if cloud test is stable

The PoC is successful if:

- Docker image builds and runs.
- Input URLs are processed.
- Invalid URLs are skipped with reasons.
- Duplicate URLs are skipped.
- Metadata is generated for each processed URL.
- Raw content is stored for successful crawls.
- Failed URLs have clear error reasons.
- The run can be repeated without corrupting output.
- Logs show progress and failures clearly.

Key metrics to review:

total input URLs
valid URLs
invalid URLs
duplicate URLs
successful crawls
failed crawls
retry count
average crawl latency
output record count

8. Release Plan

Release Stages

1. Local Docker release
 - Runs through Docker Compose.

- Processes sample input.
- Writes metadata and content locally.

2. Internal cloud PoC

- Docker image deployed to cloud.
- Small URL batch processed.
- Logs and output verified.

3. Controlled scale test

- Process 10K to 100K URLs.
- Measure throughput, failures, and cost.
- Confirm output quality.

4. Release candidate

- Documentation is complete.
 - Known blockers are documented.
 - Deployment steps are repeatable.
-

9. Quality Checklist

Before calling the PoC complete:

- No credentials are hardcoded.
 - `.env.example` is available.
 - Docker build is reproducible.
 - Output schema is documented.
 - Errors have clear reason codes.
 - Retry count is limited.
 - Logs include `batch_id` and URL hash.
 - Test run counts reconcile.
 - Cloud deployment steps are documented.
-

10. Final Recommendation

The next best step is to complete a small Docker-based PoC and deploy it to one cloud container service. I would first prove the system with 1K to 10K URLs, then move to 100K URLs once output quality and failure handling look stable.

After the PoC works, the project can move toward queue-based scaling, object storage, managed databases, dashboards, and larger monthly batches.